

Полиморфизъм

Полиморфизъм - едно име на функция, различни имплементации

Compile-time polymorphism

function overloading
operator overloading

СТАТИЧНО СВЪРЗВАНЕ - изборът коя ф-я да се извика става по време на компил-я

→ изборът е спрямо типа ун-л / реф-я, от която се вика

Runtime polymorphism

чрез виртуални ф-ии се осъществява

ДИНАМИЧНО СВЪРЗВАНЕ - изборът коя ф-я да се викне става по време на изпълнение на програмата

→ изборът е спрямо типа на обекта

Виртуална ф-я: член-функция, декларирана в Base клас, която се предефинира от Derived клас.

- Позволяват класовете да предоставят собствена имплементация на методите от базовия клас.

- не могат да са static

- Ако Der не предеф-я се ползва версията на Base

- Virtual дестр-р, но не и констр-р

КАНОНИЧЕН
АУДИО

final: - класът няма да се наследява
- ф-та няма да се презапише

override: - ф-та се презапише
- предупреждение от компилатора

Виртуални таблици

Виртуална таблица

→ таблица с ух-ли към функции

В нея са само вирт-те ф-ии и вирт-т дес-р!

В началото на всеки клас, който има вирт. функции се създава `vTable`, който сочи към виртуалната таблица на класа.

<code>A::f()</code>
<code>A::g()</code>

Примерче как изглеждат:

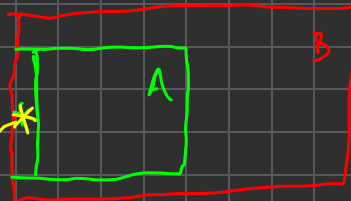
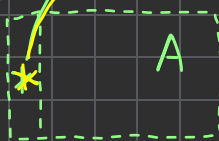
```
struct A {
```

```
    virt g()  
    virt f()  
    k();
```

```
}
```

```
B : A {
```

```
    override
```



<code>B::f()</code>
<code>A::g()</code>

При полим-та йерархия ще тръгнем об-те през ух-ли от базовия клас

=> Вирт-и дес-р там

ЧИСТО ВИРТ.

→ ф-я без тело

Абстрактен клас:

- клас с поне 1 чисто виртуална функция

→ предназначен за наследяване

→ не могат да правят обекти от него

→ ако наследник не реализира чисто вир-а ф-я => става абстрактен

забелка: вирт-та не е наследявана